

IF25-22017 PENGEMBANGAN APLIKASI MOBILE

Pertemuan 4

State dan MVVM

Membuat form laporan yang bisa diketik, tetap utuh saat layar diputar, dan logikanya tertata

Muhammad Habib Algifari, S.Kom., M.T.I.

Tahun 2026



Program Studi Teknik Informatika
Fakultas Teknologi Industri
Institut Teknologi Sumatera

Tiga jawaban dari tiket keluar minggu lalu

Pertanyaan	Jawaban singkat
Apa yang terjadi pada tampilan saat data yang dipakainya berubah?	Composable dipanggil ulang dan tampilannya ikut berubah
Kenapa padding sebelum background berbeda dengan sesudahnya?	Modifier diterapkan berurutan, jadi padding pertama menjadi jarak di luar warna
Kapan memilih LazyColumn?	Untuk daftar panjang atau yang jumlahnya tidak pasti

Form yang tidak mau diketik

Minggu lalu kita punya layar daftar. Hari ini kita membuat form laporan baru. Kodennya terlihat benar dan bisa di-build, tetapi penguji pertama langsung menemukan dua masalah seperti di samping.

DISKUSI 2 MENIT

Kenapa huruf yang diketik tidak muncul? Dan kenapa isi form hilang hanya karena layar diputar?

Laporan baru

Judul

Sudah diketik AC, tetap kosong

Lokasi

Layar diputar, isinya hilang

Kirim

Saat pulang nanti, kalian bisa

1

Mengelola state

Memakai remember dan mutableStateOf, dan menjelaskan kapan recomposition terjadi.

2

Mengangkat state

Membuat composable tanpa state yang menerima nilai dan mengirim event.

3

Menerapkan MVVM

Memindahkan state dan logika form ke ViewModel dengan StateFlow.

Capaian ini mendukung **CPMK0501**: menerapkan konsep pemrograman untuk pengembangan perangkat lunak, dalam bentuk aplikasi dengan MVVM yang berfungsi.

Bagian 1

State dan recomposition

Data yang berubah, dan cara Compose tahu kapan harus menggambar ulang.

Kode form yang membuat masalah

commonMain/ui/FormLaporan.kt

```
@Composable
fun FormLaporan() {
    var judul = "" // variabel biasa
    Column {
        TextField(
            value = judul,
            onChange = { judul = it },
            label = { Text("Judul") }
        )
        Text("${judul.length} dari 40 karakter")
    }
}
```

Kodenya bisa di-build tanpa error. Perhatikan baris `var judul = ""` dan lambda `onChange`.

Jangan jalankan dulu. Kita tebak bersama di slide berikut.

BANGUN BARENG

Kita mengetik AC di kolom judul

Apa yang tampil di kolom itu, dan berapa karakter yang tertulis di bawahnya? Tulis tebakanmu, baru kita jalankan.

remember dan mutableStateOf

commonMain/ui/FormLaporan.kt

```
@Composable
fun FormLaporan() {
    var judul by remember { mutableStateOf("") }
    Column {
        TextField(
            value = judul,
            onValueChange = { judul = it },
            label = { Text("Judul") }
        )
        Text("${judul.length} dari 40 karakter")
    }
}
```

Bagian	Tugasnya
mutableStateOf	Membuat nilai yang diawasi Compose
remember	Menyimpan nilai itu saat composable dipanggil ulang
by	Membaca dan mengubah tanpa .value

Pakai `by` butuh import `getValue` dan `setValue` dari `androidx.compose.runtime`.

Tiga cara menulis state yang sama

Penulisan	Cara mengubah	Catatan
<code>var judul by remember { mutableStateOf("") }</code>	<code>judul = "AC"</code>	Paling ringkas, kita pakai ini
<code>val judul = remember { mutableStateOf("") }</code>	<code>judul.value = "AC"</code>	Sering ada di dokumentasi lama
<code>val (judul, setJudul) = remember { mutableStateOf("") }</code>	<code>setJudul("AC")</code>	Mirip gaya React
<code>val daftar = remember { mutableStateListOf<Laporan>() }</code>	<code>daftar.add(laporan)</code>	Untuk list yang diubah isinya

Ketiga baris pertama sama persis maknanya. Pilih satu gaya dan pakai konsisten di proyek kelompok.

Recomposition: hanya yang membaca state

Recomposition: siapa yang dipanggil ulang?



Setiap huruf mengubah state judul.
Compose hanya memanggil ulang composable yang membaca judul. Sisanya dilewati.

COBA LANGSUNG



Scan atau klik untuk membaca cara Compose mengawasi state, lengkap dengan contoh kode.

[State and Jetpack Compose](#)

Sumber: Dokumentasi Android Developers, tautan dari README hands-on P4 di repo mata kuliah.

Animasi berputar saat presentasi. Di PDF yang tampil keadaan akhirnya.

Masalah kedua: layar diputar

```
commonMain/ui/FormLaporan.kt
```

```
var judul by rememberSaveable { mutableStateOf("") }
```

Saat layar diputar, Android membuat ulang Activity. Nilai yang hanya disimpan `remember` ikut hilang. `rememberSaveable` menyimpannya ke tempat yang selamat dari pembuatan ulang.

	Dipanggil ulang	Layar diputar
<code>remember</code>	Bertahan	Hilang
<code>rememberSaveable</code>	Bertahan	Bertahan, untuk nilai sederhana
<code>ViewModel</code>	Bertahan	Bertahan, ditambah logika dan data

Bagian 2

Mengangkat state

Komponen yang tidak menyimpan state sendiri lebih mudah dipakai ulang dan diuji.

Stateful dan stateless

Stateful: menyimpan sendiri

```
@Composable
fun KolomJudul() {
    var teks by remember { mutableStateOf("") }
    TextField(
        value = teks,
        onChange = { teks = it }
    )
}
```

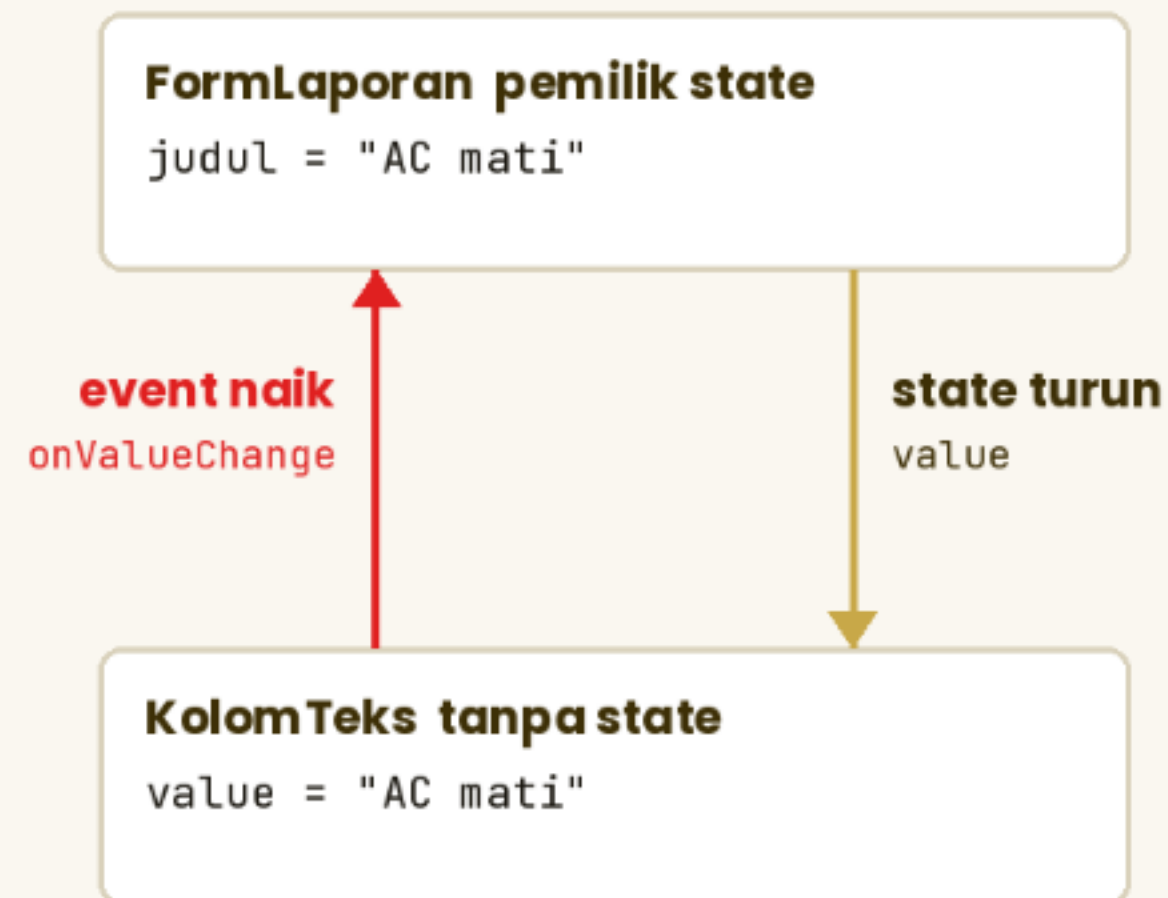
Nilainya terkunci di dalam. Parent tidak bisa membaca, mengisi, atau mengosongkannya.

Stateless: menerima dan melapor

```
@Composable
fun KolomTeks(
    label: String,
    value: String,
    onChange: (String) → Unit
) {
    OutlinedTextField(
        value = value,
        onChange = onChange,
        label = { Text(label) }
    )
}
```

State turun, event naik

State turun, event naik



Pengguna mengetik di KolomTeks, tetapi yang mengubah judul hanya FormLaporan.

COBA LANGSUNG



Scan atau klik untuk mencoba codelab resmi state dan state hoisting, langkah demi langkah.

[Codelab State in Jetpack Compose](#)

Sumber: Codelab Android Developers, tautan dari README hands-on P4 di repo mata kuliah.

Animasi berputar saat presentasi. Di PDF yang tampil keadaan akhirnya.

Satu komponen untuk semua kolom

commonMain/ui/FormLaporan.kt

```
@Composable
fun FormLaporan() {
    var judul by remember { mutableStateOf("") }
    var lokasi by remember { mutableStateOf("") }
    Column {
        KolomTeks("Judul", judul) { judul = it }
        KolomTeks("Lokasi", lokasi) { lokasi = it }
        Text("Pratinjau: $judul di $lokasi")
    }
}
```

Yang kita dapat

KolomTeks dipakai untuk judul, lokasi, dan nanti deskripsi. FormLaporan menjadi satu-satunya pemilik state, jadi pratinjau dan tombol kirim membaca sumber yang sama.

Bagian 3

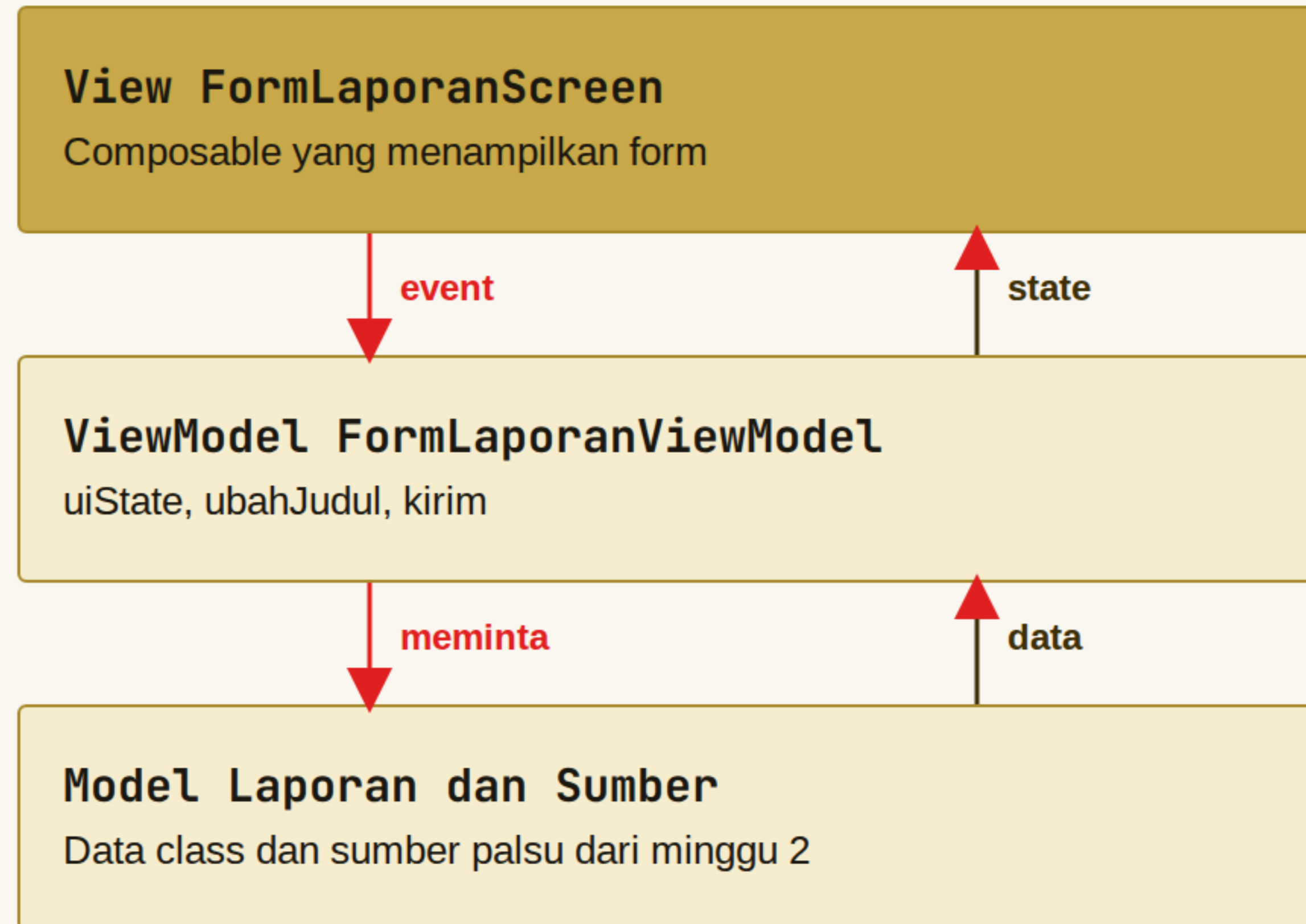
ViewModel dan MVVM

Memindahkan state dan logika form ke kelas yang bertahan saat layar diputar.

Kenapa butuh ViewModel

	State di composable	State di ViewModel
Layar diputar	Isi form hilang, kecuali rememberSaveable	Isi tetap, ViewModel tidak ikut dibuat ulang
Validasi dan aturan	Bercampur dengan kode tampilan	Di satu kelas, terpisah dari tampilan
Proses kirim	Ikut berhenti saat layar dibuat ulang	Berjalan di viewModelScope sampai selesai
Pengujian	Perlu menjalankan UI	Cukup unit test biasa, kita pakai di minggu 10

MVVM di Laporkampus



- 1 View hanya menampilkan**
Membaca uiState dan meneruskan aksi pengguna, tanpa aturan bisnis.
- 2 ViewModel memutuskan**
Menyimpan uiState, memvalidasi, dan memanggil sumber data.
- 3 Model menyimpan data**
Data class dan sumber data, nanti diganti server dan database.

Satu data class untuk seluruh layar

commonMain/ui/FormLaporanUiState.kt

```
data class FormLaporanUiState(  
    val judul: String = "",  
    val lokasi: String = "",  
    val prioritas: Int = 3,  
    val sedangMengirim: Boolean = false,  
    val pesan: String? = null  
) {  
    val bisaKirim: Boolean  
        get() = judul.isNotBlank() &&  
                lokasi.isNotBlank() && !sedangMengirim  
}
```

Semua yang dibutuhkan layar ada di satu tempat, dan setiap nilai punya bawaan.

`bisaKirim` dihitung dari nilai lain, tidak disimpan sendiri. Jadi tidak mungkin tombol aktif padahal judul kosong.

ViewModel dengan StateFlow

commonMain/ui/FormLaporanViewModel.kt

```
class FormLaporanViewModel : ViewModel() {
    private val _uiState = MutableStateFlow(FormLaporanUiState())
    val uiState = _uiState.asStateFlow()

    fun ubahJudul(teks: String) =
        _uiState.update { it.copy(judul = teks.take(40)) }
    fun ubahLokasi(teks: String) =
        _uiState.update { it.copy(lokalasi = teks) }

    fun kirim() {
        if (!_uiState.value.bisaKirim) return
        viewModelScope.launch {
            _uiState.update { it.copy(sedangMengirim = true) }
            delay(800) // minggu 6 diganti Ktor
            _uiState.value =
                FormLaporanUiState(pesan = "Laporan terkirim")
        }
    }
}
```

Bagian	Kenapa
<code>_uiState</code>	Private, hanya ViewModel yang boleh mengubah
<code>uiState</code>	Publik, layar hanya bisa membaca
<code>update { it.copy() }</code>	Mengganti objek lama dengan salinan baru
<code>viewModelScope</code>	Coroutine berhenti saat ViewModel dibersihkan

Layar yang membaca ViewModel

commonMain/ui/FormLaporanScreen.kt

```
@Composable
fun FormLaporanScreen(
    vm: FormLaporanViewModel = viewModel { FormLaporanViewModel() }
) {
    val state by vm.uiState.collectAsState()
    Column(Modifier.padding(16.dp)) {
        KolomTeks("Judul", state.judul, vm::ubahJudul)
        KolomTeks("Lokasi", state.lokasi, vm::ubahLokasi)
        Button(onClick = vm:: kirim, enabled = state.bisaKirim) {
            Text(if (state.sedangMengirim) "Mengirim" else "Kirim")
        }
        state.pesan?.let { Text(it) }
    }
}
```

Bagian	Artinya
viewModel { }	Di commonMain wajib diberi cara membuatnya
collectAsState()	Mengubah StateFlow menjadi state Compose
vm::ubahJudul	Referensi fungsi, sama dengan { vm.ubahJudul(it) }

KolomTeks dari bagian 2 dipakai tanpa diubah sedikit pun.

BANGUN BARENG

Setelah mengetik AC mati, layar kita putar

Apa isi kolom judul sekarang? Lalu, apa yang terjadi kalau tombol Kirim ditekan dua kali dengan cepat?

Cek tebakanku: ViewModel bertahan

Layar diputar: siapa yang selamat?

Hanya remember
state ikut Activity

Laporan baru

Judul

Activity dihancurkan lalu dibuat ulang, isi hilang

Dengan ViewModel
state di ViewModel

Laporan baru

Judul

ViewModel tetap hidup, isi form aman

COBA LANGSUNG



Scan atau klik, tekan Run, lalu hitung berapa kali UI menggambar. Kenapa update kedua tidak tercetak?

[Jalankan StateFlow di Kotlin Playground](#)

Sumber: Kotlin Playground, JetBrains. Bacaan lanjut: [Multiplatform ViewModel, kotlinlang.org](#)

Animasi berputar saat presentasi. Di PDF yang tampil keadaan akhirnya.

Lima kebiasaan yang kita pegang

Kebiasaan	Di kode kita
State seminimal mungkin	bisaKirim dihitung, tidak disimpan
Satu sumber kebenaran	Isi form hanya ada di FormLaporanViewModel
Hanya pemilik yang mengubah	MutableStateFlow private, StateFlow publik
Ubah dengan salinan	update { it.copy(...) }, tidak mengubah objek lama
Tanpa efek samping di composable	Kirim data lewat ViewModel, tidak di badan composable

Bagian 4

Coba sendiri, lalu bawa ke proyekmu

Tiga latihan state dan MVVM dengan observasi praktik, form tema kelompok, dan checkpoint proyek.

Yang saya amati selama latihan hari ini

Indikator RPS: **Aplikasi Notes dengan MVVM berfungsi.** Diamati dari latihan 3, Handson3TodoApp.kt.

Kriteria	Sangat baik 85 sampai 100	Baik 70 sampai 84	Cukup 55 sampai 69	Kurang di bawah 55
Aplikasi catatan dengan MVVM CPMK0501, 4%	TodoViewModel dengan StateFlow, UI hanya membaca state, tambah dan centang jalan	MVVM jalan, sebagian logika masih di composable	Hanya memakai remember, belum ada ViewModel	Aplikasi belum bisa dipakai

Individu, bobot 4% nilai akhir. Bukti: push ke repository latihan pribadimu, lalu kirim tautan commit di LMS sebelum kelas selesai.

COBA SENDIRI

Tiga latihan di repo mata kuliah



Latihan 1

Handson1Counter.kt

Counter dengan tambah, kurang yang nonaktif di nol, dan reset.

Latihan 2

Handson2RegistrationForm.kt

LabeledTextField tanpa state untuk nama dan email, lalu pratinjau.

Latihan 3

Handson3TodoApp.kt

TodoViewModel dengan StateFlow untuk menambah dan mencentang todo.

Semuanya di folder **P4 State Management MVVM Hands-on**. Menu di App.kt berpindah antara latihan dan solusi. Latihan 3 yang diamati untuk nilai praktik.

Kalau error

Gejala atau pesan	Penyebab	Yang dicoba
Tampilan tidak berubah saat diketik	Variabel biasa, bukan state	<code>remember { mutableStateOf() }</code>
Nilai kembali kosong	<code>mutableStateOf</code> tanpa <code>remember</code>	Bungkus dengan <code>remember</code>
Unresolved reference: by	Import belum ada	<code>import getValue</code> dan <code>setValue</code>
<code>collectAsState</code> not found	Import belum ada	<code>import androidx.compose.runtime.collectAsState</code>
<code>ViewModel</code> not found	Dependency belum ada	<code>lifecycle-viewmodel-compose</code> dari JetBrains

Form tambah untuk tema kelompokmu

1 **UiState form**
Data class berisi kolom entitas utama tema kalian dan bisaSimpan.

2 **ViewModel**
MutableStateFlow private, StateFlow publik, validasi sebelum simpan.

3 **KolomTeks dipakai ulang**
Satu composable tanpa state untuk semua kolom isian.

4 **Daftar ikut bertambah**
Sumber palsu menyimpan daftar dalam MutableStateFlow.

CONTOH DARI LAPORKAMPUS

FormLaporanUiState, FormLaporanViewModel, dan KolomTeks. Laporan baru langsung muncul di layar daftar minggu 3.

Checkpoint proyek minggu 4

1 Form tambah
Layar form entitas utama tema, memakai KolomTeks tanpa state.

2 ViewModel
UiState dan ViewModel dengan StateFlow, simpan hanya bila valid.

3 Daftar ikut bertambah
Data baru masuk sumber palsu dan tampil di layar daftar.

Dikumpulkan

Push ke repository kelompok. README memuat tangkapan layar form dan daftar setelah data ditambah.

Dicek di awal pertemuan berikutnya: tuntas, sebagian, atau belum.

TUTUP

Tiket keluar



1 State

Kenapa `var judul = ""` di dalam composable membuat kolom tidak bisa diketik?

2 Hoisting

Apa yang dikirim parent ke `KolomTeks`, dan apa yang dikirim `KolomTeks` ke parent?

3 MVVM

Kenapa `uiState` yang publik bertipe `StateFlow`, bukan `MutableStateFlow`?

Jawab singkat di kertas atau formulir LMS sebelum keluar kelas.

TUTUP

Minggu depan: navigasi antar layar



MASALAH MINGGU 5

Daftar dan form sudah ada. Bagaimana pindah dari daftar ke detail satu laporan sambil membawa id-nya, lalu kembali tanpa kehilangan posisi gulir?

Persiapan

Bawa form dan daftar kelompok kalian. Pastikan TodoApp latihan 3 sudah di-push ke repository latihan pribadi.

Terima kasih

Sampai jumpa di pertemuan 5

Ada pertanyaan? Sampaikan sekarang, lewat LMS, atau email ke muhammad.algifari@if.itera.ac.id